3. METODOLOGÍA

3.1 Enfoque metodológico y criterios de decisión

El desarrollo de *The Show Verse* se ha abordado mediante una **metodología iterativa basada en el ciclo de vida ágil**, adaptada a un proyecto individual. La elección de este enfoque frente a las metodologías en cascada se justifica por la naturaleza exploratoria del proyecto: la integración con APIs de terceros implica incertidumbres técnicas que no siempre se pueden anticipar en una fase de análisis, y la prioridad entre funcionalidades puede variar a medida que se comprende mejor el problema.

El proceso de desarrollo se articula en iteraciones de aproximadamente una semana de duración. Al inicio de cada iteración se seleccionan del backlog las funcionalidades a abordar, priorizadas según tres criterios:
- **Dependencia técnica:** las funcionalidades que sirven de base a otras (autenticación, cliente de API) se abordan primeras.
- **Valor aportado al usuario:** se priorizan las funcionalidades más visibles y relevantes para la experiencia de uso.
- **Riesgo técnico:** las integraciones más inciertas se adelantan para conocer cuanto antes sus restricciones.

Criterios de integración de APIs La selección de las APIs externas ha seguido los siguientes criterios:

- **Datos completos:** TMDb fue seleccionada frente a otras alternativas (The Open Movie Database, Wikidata) por ofrecer la mayor cobertura de contenido, soporte en diferentes idiomas, galería de imágenes y datos de streaming. Dispone de la documentación más completa y una comunidad de desarrolladores activa.

- **Capacidad de tracking:** Trakt.tv fue seleccionada como plataforma de sincronización por su API pública v2 bien documentada, su soporte completo de series a nivel de episodio, la disponibilidad de sus endpoints principales para desarrolladores, y por ser la plataforma de *tracking* más referenciada por la comunidad de desarrolladores independientes.

- **Complementariedad:** OMDb fue seleccionada únicamente para los datos de rating de IMDb y Rotten Tomatoes, que TMDb no proporciona directamente.

Criterios de decisión para la autenticación El problema de la autenticación OAuth 2.0 admitía varias opciones de implementación en Next.js. Se descartó el uso de librerías de autenticación genéricas como NextAuth.js / Auth.js por las siguientes razones: - Trakt.tv no es un proveedor OAuth soportado de forma nativa por estas librerías. - La personalización necesaria (refresco de tokens específico de Trakt, intercambio de códigos con headers personalizados)

habría requerido más configuración que implementar el flujo directamente. - La implementación propia ofrece mayor control sobre el manejo de errores y el almacenamiento de tokens.

Se optó por implementar el flujo Authorization Code de forma manual mediante **API Routes de Next.js**, almacenando los tokens en cookies `httpOnly` con atributos `Secure` y `SameSite=Lax`. Esta decisión garantiza que los tokens no sean accesibles desde JavaScript del navegador (protección frente a XSS) y que no se envíen en peticiones *cross-site* no intencionadas (protección parcial frente a CSRF).

Criterios de diseño de interfaz Las decisiones de diseño visual han seguido los siguientes principios: - **Coherencia con el dominio:** una plataforma de contenido audiovisual compite visualmente con las propias plataformas de *streaming*, desde el punto de vista del usuario, lo que justifica una estética similar. - **Modo oscuro por defecto:** adecuado para el consumo de contenido multimedia y preferido por la mayoría de usuarios. - **Mobile-first:** el consumo de contenido audiovisual en dispositivos móviles supera al escritorio en grupos demográficos menores de 35 años según datos de Nielsen (2023).

Criterios de estrategia de caché Para cada tipo de dato gestionado en la aplicación se ha definido una estrategia de caché en función de la frecuencia de actualización del dato en origen:

Tipo de dato	Almacenamiento	Revalidación
Catálogo general (top rated, populares)	ISR servidor (force-cache)	10 minutos
Detalles de película/serie	ISR servidor (force-cache)	1 hora
Imágenes TMDb	CDN Vercel	1 hora
Estado personal del usuario (favoritos, historial)	No-cache	En cada petición
Datos de autenticación (token)	Cookie httpOnly	Por expiración
Puntuaciones IMDb y Trakt en listas	**localStorage (cliente)**	**30 días**
Datos OMDb en página de detalle (IMDb rating, RT, Metacritic)	**sessionStorage (cliente)**	**24 horas**
Rating personal del usuario en página de detalle	**Memoria (Map en módulo)**	**10 min (nulo: 45 s)**

Esta diferenciación permite reducir el número de llamadas a APIs externas en un estimado del 70–80% para los datos de catálogo, mientras se garantiza la actualización de los datos de usuario. El nivel de caché en cliente dedicado a las puntuaciones de fuentes externas (IMDb, Trakt, OMDb) reduce adicionalmente la carga sobre estas APIs en las vistas de listas y detalles, donde se consultan simultáneamente puntuaciones de múltiples títulos.

3.2 Tecnologías empleadas

Next.js 16 (Framework principal)
Seleccionado frente a otras alternativas (Remix, SvelteKit, Nuxt 3) por ser el framework React con mayor adopción industrial (>40% de cuota entre frameworks React según npm trends 2024), la mejor integración con Vercel (plataforma de despliegue objetivo) y la documentación y comunidad más extensa. El App Router de Next.js 13+ es la única opción que implementa React Server Components de forma estable y con soporte oficial.

React 19
La elección de React como biblioteca de UI es consecuencia directa de la elección de Next.js. React 19 introduce mejoras en el manejo de acciones de servidor y la promesa nativa en componentes de cliente, aunque en este proyecto se utiliza primariamente en su versión estable de componentes de cliente e hidratación.

Tailwind CSS 4
Seleccionado frente a CSS Modules, Styled Components o Emotion por el paradigma *utility-first* que acelera el desarrollo sin sacrificar la personalización. La versión 4, basada en PostCSS con motor Rust nativo, elimina el *PurgeCSS* separado e incorpora mejoras sustanciales de rendimiento en el tiempo de compilación.

Framer Motion 12
La elección de Framer Motion frente a otras librerías de animación (React Spring, CSS Animations puras) se basa en su modelo declarativo basado en variantes, que permite expresar animaciones complejas de forma legible, su soporte nativo para animaciones de *layout* (el *Flip Animation Technique*), y su integración con el ciclo de vida de los componentes React mediante `AnimatePresence`.

TypeScript 5
Configurado en modo de verificación estricta para la configuración del proyecto y los tipos de datos de las respuestas de API. En los componentes React se utiliza JavaScript con JSX, manteniendo la flexibilidad de tipado dinámico en la lógica de componentes.

---

4. DESARROLLO

4.1 Arquitectura del sistema

El sistema se estructura en tres capas diferenciadas, implementadas en una única base de código Next.js:

Capa de presentación (Client Components)
Componentes React que se ejecutan y se hidratan en el navegador. Gestionan el estado local de la interfaz, las interacciones del usuario y las peticiones a la capa intermedia. Los componentes de mayor complejidad son `MainDashboardClient.jsx` (gestión del dashboard completo), `DetailsClient.jsx` (página de detalle de película y serie) y `DiscoverClient.jsx` (módulo de búsqueda).

Capa de servidor (Server Components + API Routes)
Los Server Components de Next.js realizan la búsqueda inicial de datos directamente en el servidor, sin coste de JavaScript para el cliente y las API Routes actúan como proxy hacia las APIs externas, añadiendo los tokens de autenticación en el servidor y transformando los datos al formato que consume el frontend.

Capa de datos (APIs externas)
TMDb, Trakt.tv y OMDb como fuentes para los datos de catálogo y del usuario respectivamente.

La separación entre estas capas se hace explícita en la convención de nombres del proyecto: los archivos terminados en `Client.jsx` son siempre Client Components (marcados con `"use client"`), mientras que los archivos `page.jsx` y los de la carpeta `api/` son siempre Server Components o API Routes.

4.2 Implementación del sistema de autenticación OAuth 2.0

La implementación del flujo OAuth 2.0 con Trakt.tv es el núcleo técnico más delicado del proyecto. El flujo se articula en cuatro API Routes de Next.js:

Inicio del flujo (`/api/trakt/oauth`): construye la URL de autorización de Trakt con los parámetros `client_id`, `redirect_uri` y `response_type=code`, y redirige al usuario. Este endpoint se invoca al pulsar el botón "Conectar con Trakt" de la Navbar.

Callback (`/api/trakt/auth/callback`): recibe el `authorization_code` de Trakt, lo intercambia por `access_token` y `refresh_token` mediante una petición POST servidor-a-servidor a `https://api.trakt.tv/oauth/token`, y almacena ambos tokens en cookies `httpOnly`. A continuación redirige al usuario a la página de inicio.

Estado (`/api/trakt/auth/status`): lee las cookies de sesión y, si existe `access_token` válido, hace una petición a `/users/me` de Trakt para obtener

el perfil del usuario. Si el token está expirado, intenta refrescarlo usando el `refresh_token` antes de declarar al usuario como no autenticado.

Desconexión (`/api/trakt/auth/disconnect`): revoca el token en Trakt mediante la API `/oauth/revoke` y elimina las cookies de sesión.

Este diseño garantiza que ni el `client_secret` de Trakt ni los `access_token` del usuario sean accesibles desde el JavaScript del navegador en ningún momento del flujo.

4.3 Integración con las APIs externas

Cliente TMDb El cliente de TMDb (`tmdb.js`) encapsula toda la lógica de acceso a la API v3. La función central `tmdb(path, params, options)` implementa:

- Un **constructor de URL unificado** que añade automáticamente la API Key y el idioma por defecto (`es-ES`).
- Un **mecanismo de timeout** mediante `AbortController` (8 segundos por defecto en cliente, ajustable por opción) para evitar que peticiones lentas bloqueen la interfaz.
- **Comportamiento de caché diferenciado** según el entorno: `force-cache` con revalidación ISR en servidor, `no-store` en cliente (el navegador gestiona su propia caché HTTP).
- **Gestión granular de errores:** los errores 404 retornan `null` silenciosamente (contenido no encontrado es un caso esperado); otros errores se loguean y también retornan `null` para que el componente gestione el estado de error.

Sobre esta función base se construyen más de 40 funciones nombradas por dominio (`fetchTopRatedMovies`, `getDetails`, `getCredits`, `getWatchProviders`, `getActorDetails`, etc.) que constituyen la API pública del módulo.

Un aspecto técnico importante es la resolución de imágenes: la función `getLogos` obtiene el logo oficial de un título (endpoint `/images` de TMDb) aplicando un criterio de selección: prioriza logos en español, luego en inglés, luego sin idioma; dentro de cada grupo selecciona el de mayor número de votos. Este logo se usa como título visual en la página de detalle cuando está disponible, en lugar del texto plano del título.

Proxy de Trakt y transformación de datos Las API Routes de Trakt actúan como un intermediario que resuelve dos problemas estructurales:

Problema de identificadores: el frontend trabaja exclusivamente con `tmdbId` (identificadores de TMDb), mientras que muchos endpoints de Trakt requieren sus propios identificadores (`traktId`, `slug`). Las API Routes realizan internamente la traducción mediante el endpoint de búsqueda por ID externo de Trakt (`/search/tmdb/{id}?type=movie|show`).

Problema de enriquecimiento de datos: las respuestas de Trakt incluyen datos de *tracking* pero no imágenes. Las API Routes que devuelven listas de contenido (historial, favoritos, pendientes) realizan peticiones paralelas a TMDb para obtener las URLs de poster y backdrop, enriqueciendo la respuesta antes de enviarla al cliente.

El cliente del frontend (`traktClient.js`) expone una API orientada a operaciones de usuario: `traktSetWatched`, `traktSetEpisodeWatched`, `traktHistoryOp`, `traktSetRating`, etc. Internamente, todas estas funciones realizan peticiones fetch a las API Routes internas, que son las que contienen el token de autenticación y la lógica de llamada a Trakt.

Una particularidad técnica de este módulo es el manejo de fechas: Trakt acepta formatos distintos según el endpoint (ISO 8601 completo para el historial de *plays*; `YYYY-MM-DD` para favoritos y *watchlist*). El cliente incluye las funciones `normalizeWatchedAtForHistoryApi` y `normalizeWatchedAtForApi` que normalizan cualquier entrada (objeto `Date`, string en varios formatos) al formato correcto para cada caso.

Caché de puntuaciones en listas de favoritos y pendientes Las páginas de Favoritos (`FavoritesClient.jsx`) y Pendientes(`WatchlistClient.jsx`) muestran, por cada ítem de la lista, las puntuaciones IMDb y Trakt de la comunidad junto con la valoración personal del usuario en IMDb y Trakt. Estas puntuaciones se obtienen de las APIs externas (OMDb, Trakt *scoreboard* endpoint) y deben cargarse de forma asíncrona y paginada para no exceder los límites de peticiones.

Para evitar peticiones redundantes entre sesiones y navegaciones, se implementa un **sistema de caché de puntuaciones en tres niveles**:

Nivel 1 — `localStorage` con TTL de 30 días (puntuaciones IMDb y Trakt). Las funciones `readScoreCache(source)`, `writeScoreCache(source, map)` y `updateScoreCache(source, id, score)` gestionan un diccionario persistente en `localStorage` bajo las claves `showverse:scores:imdb` y `showverse:scores:trakt`. Cada entrada almacena el valor numérico y el *timestamp* de escritura (`{ score, t }`). En la lectura, las entradas con antigüedad superior a 30 días se descartan automáticamente. Este TTL largo está justificado porque las puntuaciones de comunidad de IMDb y Trakt cambian con muy baja frecuencia.

Nivel 2 — `sessionStorage` con TTL de 24 horas (datos OMDb). El módulo `omdbCache.js` gestiona una caché en `sessionStorage` bajo la clave `showverse:omdb:{imdbId}`, que almacena el rating de IMDb, los votos, los premios y las puntuaciones de Rotten Tomatoes y Metacritic obtenidos de la API de OMDb. Este mismo módulo se reutiliza en la página de detalle y en la lista de favoritos para evitar llamadas duplicadas a OMDb durante una misma sesión de navegación.

Nivel 3 — Caché en memoria (`Map`) con TTL corto (rating personal del usuario). Los mapas `userRatingCache` y `traktScoreCache` (instancias de `Map` a nivel de módulo) almacenan en memoria las valoraciones personales del usuario autenticado y las puntuaciones de comunidad de Trakt, con TTL de 10 minutos para entradas con valor y 45 segundos para entradas nulas (para reintentar si la API no respondió). Estos TTL cortos garantizan que los datos de usuario se actualizan con frecuencia razonable, mientras se evitan peticiones en cada *re-render*.

Carga diferida y patrón *on-hover*. Para minimizar el número de peticiones en el montaje inicial del componente, las puntuaciones IMDb y Trakt se cargan de forma diferida al primer *hover* sobre una tarjeta de la lista (`handleHover`). Si el valor ya está en caché de nivel 1, se recupera instantáneamente sin llamada de red. La función `runPool(items, limit, worker)` controla la concurrencia máxima de peticiones simultáneas para respetar los *rate limits* de las APIs externas.

Barra de puntuaciones en página de detalle (`ScoreboardBar`) El componente `ScoreboardBar.jsx` centraliza la visualización de puntuaciones en la página de detalle de película o serie: puntuación TMDb (procedente de los datos ya cargados por el Server Component, sin petición adicional), puntuación de comunidad de Trakt (obtenida del endpoint `/api/trakt/scoreboard`) y rating de IMDb (procedente de OMDb a través de `omdbCache`). El componente también gestiona la valoración personal del usuario en Trakt (estrellas 1–10), con actualización optimista del estado local.

4.4 Implementación de los módulos funcionales principales

Dashboard principal La página de inicio (`/`) es un Server Component que obtiene en paralelo los datos de todas las secciones mediante `Promise.all`, reduciendo el tiempo total de carga a la latencia de la llamada más lenta en lugar de la suma de todas ellas. Para el hero, se aplica una selección inteligente del backdrop: se descartan imágenes sin texto en inglés (inadecuadas para el hero), se priorizan las de mayor resolución (1280px) y, dentro de estas, las de mayor número de votos.

El componente cliente `MainDashboardClient.jsx` recibe todos los datos del servidor ya preparados y se encarga exclusivamente de la composición visual y las interacciones: el carrusel automático del hero (con pausa al hover), el sticky scroll de la Navbar y la aparición progresiva de las secciones mediante `staggerChildren` de Framer Motion.

Seguimiento por episodio El módulo de episodios es el funcionalmente más complejo. El estado de visionado por episodio se representa mediante el objeto `watchedBySeason`.

Este objeto se obtiene del endpoint `/api/trakt/show/watched`, que hace una

llamada a `/sync/watched/shows` de Trakt y transforma la respuesta para indexarla por temporada y número de episodio. Cada toggle de episodio actualiza este objeto optimistamente en el estado local del componente, mientras la petición al servidor se procesa en segundo plano.

El botón "Marcar temporada completa" construye la lista de episodios no vistos de la temporada y realiza una única llamada a Trakt, reduciendo el número de peticiones de N a 1.

Actualización de la interfaz Para las operaciones de favoritos, pendientes e historial de visionado, se implementa el siguiente patrón de actualización de la interfaz:

1. Cuando el usuario realiza una acción, el estado local se actualiza inmediatamente (el botón cambia de estado, el contador de la Navbar se actualiza).
2. En paralelo, se envía la petición al servidor.
3. Si la petición falla, el estado local se revierte al valor anterior y se muestra una notificación de error.

Este patrón elimina el retraso perceptible entre la acción del usuario y la respuesta de la interfaz, mejorando significativamente la experiencia de usuario.

4.5 Diseño de la interfaz y sistema de vistas

La interfaz implementa el sistema de diseño descrito en la metodología. El fondo negro (`#000000`) como color base crea el lienzo sobre el que los fondos (*backdrops*) de las películas y series actúan como elemento visual protagonista. Las superficies de las tarjetas y los paneles laterales utilizan valores de opacidad muy bajos (`rgba(255,255,255,0.05)`) para el efecto visual, con bordes de `1px solid rgba(255,255,255,0.1)`.

El sistema de tres vistas (Grid, List, Compact) es un componente transversal que se aplica en varios módulos de la aplicación (Favoritos, Pendientes, Historial, En Progreso). La selección de vista se persiste en `localStorage` por módulo, de forma que el usuario puede tener configuraciones distintas en cada sección. La transición entre vistas utiliza `AnimatePresence` de Framer Motion con un efecto de *cross-fade* suave, evitando saltos bruscos en el *layout*.

4.6 Rendimiento en producción

Las métricas obtenidas en producción (Vercel) mediante Lighthouse validan las decisiones de arquitectura tomadas:

Métrica	Valor	Objetivo
Performance (Lighthouse)	92/100	> 90
SEO (Lighthouse)	100/100	100
Best Practices	95/100	> 90

Métrica	Valor	Objetivo
LCP	1.8s	< 2.5s
CLS	0.05	< 0.1
FID	45ms	< 100ms

El LCP de 1.8s se consigue principalmente gracias al SSR: el HTML con el hero ya renderizado llega al navegador antes de que se descargue todo el JavaScript, permitiendo que el motor de renderizado del navegador comience a mostrar contenido de inmediato. El CLS de 0.05 se logra mediante el uso sistemático de `width` y `height` explícitos en el componente `<Image>` de Next.js, que reserva el espacio de la imagen antes de que esta se descargue.

---

5. CONCLUSIONES

El trabajo ha demostrado que los objetivos planteados son alcanzables con las tecnologías actuales de desarrollo web y sin necesidad de infraestructura de servidor propia. Los resultados obtenidos permiten extraer las siguientes conclusiones:

En primer lugar, la arquitectura Next.js App Router con React Server Components ha demostrado ser particularmente adecuada para aplicaciones de este tipo, que combinan una gran cantidad de datos externos con la necesidad de interactividad en cliente. La separación entre la obtención de datos (servidor) y la presentación interactiva (cliente) simplifica el flujo de datos y reduce las fuentes de error. Los Server Components permiten realizar fetching paralelo de múltiples APIs en el servidor sin coste para el cliente, lo que resulta en tiempos de carga significativamente inferiores a los de una SPA equivalente.

En segundo lugar, la integración de OAuth 2.0 mediante API Routes de Next.js ha probado ser una estrategia robusta y segura sin necesidad de librerías de autenticación genéricas. El patrón de almacenar tokens en cookies `httpOnly` y delegar toda la lógica de autenticación al servidor cumple con las recomendaciones de seguridad del RFC 6749 y las guías de OWASP para aplicaciones OAuth.

En tercer lugar, la estrategia de caché diferenciada por tipo de dato ha demostrado ser eficaz en todas sus capas: ISR en servidor para el catálogo (servido desde caché en prácticamente el 100% de las peticiones), `no-store` para los datos de usuario protegidos por autenticación, y una caché en cliente de tres niveles para las puntuaciones externas de IMDb, Trakt y OMDb en las vistas de listas. Esta última capa —`localStorage` con TTL de 30 días para puntuaciones de comunidad, `sessionStorage` con TTL de 24 horas para datos de OMDb y caché en memoria para el rating personal del usuario— reduce drásticamente el número de llamadas a APIs externas en las páginas de Favoritos y *Watchlist*, donde se consultan simultáneamente puntuaciones de decenas de títulos.

Finalmente, desde el punto de vista del diseño, el uso de Framer Motion para las animaciones y el sistema de superficies semi-transparentes con desenfoque de fondo ha producido una interfaz visualmente diferenciada respecto a las plataformas de *tracking* existentes, más próxima en estética a las propias plataformas de *streaming*. Las métricas de rendimiento obtenidas son completamente compatibles con un diseño visualmente rico, siempre que las animaciones se implementen sobre la GPU (propiedades `transform` y `opacity`) y no fuercen la recarga del DOM.

5.1 Discusión

El trabajo presenta algunas limitaciones que deben reconocerse:

Ausencia de tests automatizados. El proyecto no incluyó una suite de tests unitarios ni de integración durante el desarrollo. La validación se realizó exclusivamente mediante pruebas manuales y esta decisión, justificada por las restricciones temporales del TFG, supone el principal déficit de calidad del proyecto desde el punto de vista del proceso de ingeniería del software. Los componentes con lógica de negocio compleja (cálculo de progreso de episodios, normalización de fechas para Trakt, estrategia de selección de backdrops) serían candidatos prioritarios para la cobertura de tests unitarios.

Dependencia de APIs de terceros. La plataforma depende de la continuidad y estabilidad de tres APIs externas (TMDb, Trakt.tv, OMDb), ninguna de las cuales está bajo el control del proyecto. Cambios en sus modelos de precios, modificaciones de sus límites de peticiones o interrupciones de servicio afectarían directamente a la aplicación. Esta es una limitación estructural de la arquitectura elegida, aceptada conscientemente por las ventajas que ofrece frente a la alternativa de construir una base de datos propia.

Accesibilidad parcial. La puntuación de accesibilidad en Lighthouse (88/100) indica que, aunque se han aplicado prácticas básicas (textos alternativos en imágenes, contraste de colores suficiente), no se ha alcanzado el nivel WCAG 2.1 AA completo. En particular, la navegación exclusiva por teclado en los modales de episodios y el manejo del foco en las transiciones de página requieren trabajo adicional.

Ausencia de gestión de usuarios propia. La plataforma delega toda la gestión de identidad en IMDb y Trakt.tv, lo que significa que los usuarios deben tener una cuenta en Trakt para acceder a las funciones personalizadas. Aunque esto reduce la complejidad de implementación, limita la autonomía de la plataforma.

5.2 Líneas de investigación futuras

A partir del trabajo realizado, se identifican las siguientes líneas de desarrollo futuro:

Implementación de Progressive Web App (PWA). La conversión de la aplicación en una PWA, mediante la adición de un *Service Worker* y un *manifest* web, permitiría la instalación en dispositivos móviles y escritorio, y habilitaría un modo de funcionamiento *offline* básico (visualización del historial en caché). Esta línea es de especial interés dado que el consumo de contenido audiovisual está fuertemente correlacionado con el dispositivo móvil.

Sistema de recomendaciones personalizado. El historial de visionado acumulado por el usuario constituye un conjunto de datos con potencial para la implementación de algoritmos de recomendación colaborativa o basada en contenido. La integración de un modelo de *machine learning* ligero (posiblemente mediante TensorFlow.js en el cliente o una función *serverless*) para enriquecer las secciones del dashboard con recomendaciones verdaderamente personalizadas es una extensión natural del trabajo actual.

Expansión de las funcionalidades sociales. La comparación del historial entre usuarios, las recomendaciones de amigos y la visualización del perfil público de otros usuarios de Trakt son funcionalidades que la API de Trakt soporta pero que no han sido implementadas en la versión actual.

Backend propio y base de datos local. A largo plazo, la eliminación de la dependencia total de Trakt.tv como fuente de verdad para los datos del usuario requeriría la implementación de un backend propio con base de datos (PostgreSQL o MySQL) que almacenara el historial y las preferencias localmente. Esto permitiría funcionalidades no disponibles en Trakt (estadísticas avanzadas, exportación de datos en formatos personalizados) y eliminaría la dependencia de servicio externo.

Integración con plataformas de streaming. Las APIs de Netflix, Disney+ y Amazon Prime no son públicas, pero servicios como JustWatch disponen de datos actualizados sobre disponibilidad de contenido en cada plataforma y país. La integración de esta información en la página de detalle permitiría completar el ciclo de usuario: "¿Dónde puedo ver este contenido ahora mismo?".

---

6. REFERENCIAS

6.1 Bibliografía

Babich, N. (2021). *Glassmorphism in User Interfaces.* UX Planet. https://uxplanet.org/glassmorphism-in-user-interfaces-1f39bb1308c9

Berners-Lee, T., Fielding, R., y Masinter, L. (1999). *RFC 2396: Uniform Resource Identifiers (URI): Generic Syntax.* IETF. https://www.rfc-editor.org/rfc/rfc2396

Bloch, J. (2008). *Effective Java* (2.ª ed.). Addison-Wesley.

CNMC. (2024). *Informe sobre el consumo de vídeo en España.* Comisión Nacional de Mercados y la Competencia. https://www.cnmc.es/

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Framer. (2024). *Framer Motion Documentation.* https://www.framer.com/motion/

Google Developers. (2024). *Web Vitals — Essential metrics for a healthy site.* https://web.dev/vitals/

Google Developers. (2024). *Core Web Vitals report.* https://support.google.com/webmasters/answer/9205520

Hardt, D. (Ed.). (2012). *RFC 6749: The OAuth 2.0 Authorization Framework.* IETF. https://www.rfc-editor.org/rfc/rfc6749

Letterboxd. (2024). *About Letterboxd.* https://letterboxd.com/about/

Lotz, A. D. (2022). *Netflix and Streaming Video: The Business of Subscriber-Funded Video on Demand.* Polity Press.

Meta Open Source. (2024). *React Documentation — React Server Components.* https://react.dev/reference/rsc/server-components

Nielsen. (2023). *The Nielsen Streaming Unwrapped: 2023 Mid-Year Report.* Nielsen. https://www.nielsen.com/insights/2023/streaming-unwrapped/

OASIs. (2019). *OpenAPI Specification 3.0.* https://spec.openapis.org/oas/v3.0.0

OWASP. (2024). *OWASP Top 10: Broken Authentication.* Open Web Application Security Project. https://owasp.org/www-project-top-ten/

Schwartz, B. (2004). *The Paradox of Choice: Why More Is Less.* Ecco/HarperCollins.

Statista. (2024). *Number of Netflix subscribers worldwide from 1st quarter 2013 to 4th quarter 2023.* https://www.statista.com/statistics/250934/quarterly-number-of-netflix-streaming-subscribers-worldwide/

The Movie Database. (2024). *TMDb API Documentation v3.* https://developer.themoviedb.org/docs/getting-started

Trakt. (2024). *Trakt API Documentation v2.* https://trakt.docs.apiary.io/

Vercel. (2024). *Next.js Documentation — App Router.* https://nextjs.org/docs/app

Vercel. (2024). *Vercel Documentation — Edge Functions and Middleware.* https://vercel.com/docs/functions

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1.* World Wide Web Consortium. https://www.w3.org/TR/WCAG21/

Wieruch, R. (2023). *The Road to Next.js.* Robin Wieruch. https://www.roadtonextjs.com/